An open, containerized pipeline turning multi-image captures — from consumer 360° cameras to terrestrial laser scanners — into web-navigable 3D scenes

Author: Angel Naya, Independent Researcher, Berlin, Germany
Corresponding author: Angel Naya, angel.naya@posteo.de
Target journal: SoftwareX (Elsevier) — Original Software Publication

Abstract

Walkable 360° panoramas and point clouds are valuable for documentation, monitoring, and outreach, yet their production is usually tied to proprietary, cloud-bound toolchains. We present a fully open-source, containerized processing chain with a graphical user interface that turns multi-image captures into walkable, web-based 3D scenes. The chain unifies two input classes that are normally handled separately: (i) captures without known pose (consumer and DSLR fisheye), assembled by stitching (Hugin), and (ii) the RGB images of terrestrial laser scanners, whose poses are stored in the E57 file and can therefore be reprojected directly — a seldom-used by-product. Output is served by an open-source web viewer (Pannellum, three.js), self-hostable through Caddy and Garage, fully packaged in Docker. In an evaluation against eleven CC0 reference panoramas, the pose-based branch outperforms stitching on every measure (26.9 vs 23.0 dB PSNR, 0.886 vs 0.837 SSIM, 0.10 vs 0.14 px seam offset), reconstructs all eleven scenes versus six, and does so roughly seven times faster. Both branches run fully automatically without manual control points. The software, container recipe, and evaluation code are openly available; all example data are third-party open data (CC BY 4.0, MIT, CC0).

Keywords: panorama stitching; 360° imaging; terrestrial laser scanning; E57; open-source; self-hosting; Docker; web visualization; photogrammetry

Code metadata

Current code version	v1.0
Permanent link to code/repository	https://github.com/anayana/For3Dsuite
Permanent link to reproducible capsule	https://doi.org/10.5281/zenodo.23323560
Legal code license	GNU General Public License v3.0 or later (GPL-3.0-or-later)
Code versioning system	git
Languages / tools	Python, JavaScript; Hugin, GIMP, Pannellum, three.js, Caddy, Garage, Docker
Compilation / runtime	Docker + Compose (single command)
Link to developer documentation	https://github.com/anayana/

	For3Dsuite#readme (platform/README.md)
Support e-mail	angel.naya@posteo.de

1. Motivation and significance

Interactive 360° panoramas and 3D point clouds are increasingly used to document and communicate spatial states — in forest and environmental monitoring, surveying, cultural heritage, and teaching. Two obstacles recur in practice. First, the common routes are either proprietary and cloud-bound (camera apps with vendor hosting) or fragmented: a stitching tool, a web viewer, and a server must be wired together by hand, with no reproducible, containerized path from raw frames to a published, walkable scene. Second, a valuable data source is routinely discarded: terrestrial laser scanners (TLS) acquire RGB images whose camera poses are stored alongside the geometry in the E57 exchange format, yet these images are usually treated only as a means to colourize the point cloud or, as panoramic reflectance images, to register scans [6, 7] — not as navigable panoramas in their own right.

We address both with a single open-source tool. Its contribution is not any individual component — panorama stitching [1, 2], Pannellum [3], and GIMP are established — but their **integration into one automated, containerized, GUI-driven, vendor-agnostic chain**, and specifically the **unification of the two input classes**: captures whose pose must be estimated (consumer/DSLR fisheye) and captures whose pose is already known (scanner RGB in E57 [5]). The former are stitched; the latter are reprojected directly to the equirectangular domain, which is faster and free of stitching failure modes. The same output can be enriched with data derived from the point cloud (stem detection, diameter at breast height, height, crown metrics, quantitative structure models, growth projection) and served fully self-hosted, giving data sovereignty without a cloud dependency.

The tool spans the cost and device spectrum — from a ~20 € consumer 360° camera to a six-figure TLS — into the same web scene, lowering the barrier for reproducible spatial documentation.

2. Software description

2.1 Software architecture

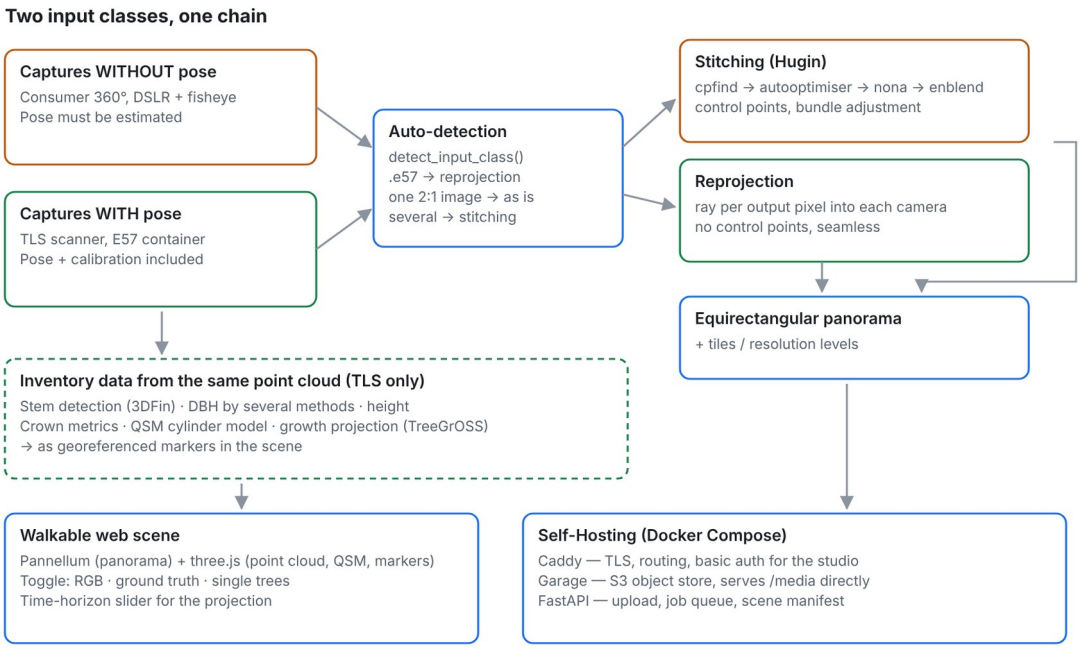


Figure 1. *Two input classes, one chain. The input class is determined from the data (`detect_input_class`), not declared by the user: an `.e57` container carries pose and is reprojected; a single 2:1 image is taken over as is; several images without pose are stitched. For TLS input, inventory derived from the same point cloud is attached to the scene as georeferenced markers.*

The system is packaged as a set of Docker services orchestrated by Compose: a processing/API service (Python, FastAPI job queue), an S3-compatible object store (Garage), a reverse proxy with automatic TLS (Caddy), and a browser-based studio GUI. A single command brings up a reproducible stack.

At ingestion the tool determines the input class automatically (`detect_input_class`): an uploaded `.e57` triggers **reprojection**; a single image with a 2:1 aspect ratio is accepted as a finished equirectangular panorama; several images trigger **stitching**. A single non-2:1 image is deliberately rejected rather than guessed, so the tool never silently mis-handles ambiguous input. In the container, the same upload endpoint without a declared type correctly recognizes `equirect` (one 2:1 image) and `fisheye` (six frames) and runs the corresponding chain through to a published scene.

2.2 Software functionalities

- **Stitching branch** (pose unknown): Hugin/Panotools with automatic control-point detection (`cpfind`, `cpclean`), optimization (`autooptimiser`), and seam blending (`nona`, `enblend`); no manual control points.

- **Reprojection branch** (pose known): the per-image poses from the E57 are used to resample the RGB images into a seamless equirectangular panorama (bilinear sampling).
- **Pre-processing**: batch exposure/crop/nadir–zenith handling (GIMP Script-Fu).
- **Viewing**: web-based panorama and point-cloud viewer (Pannellum [3], three.js), multi-resolution output; for very large clouds, dedicated streaming renderers such as Potree [4] remain the reference.
- **Enrichment**: georeferenced markers carrying point-cloud-derived inventory data overlaid on the walkable scene.
- **Publishing**: self-hosted (Caddy + Garage) or as a static export.

3. Illustrative examples

Scanner RGB → walkable scene with data (Renon / ICOS IT-Ren). A TLS E57 with six pinhole images and poses per setup is reprojected to a seamless equirectangular panorama; the co-registered point cloud is added, and inventory markers (stem detection, DBH, height, crown metrics, quantitative structure model, growth projection) are placed in the walkable scene. This demonstrates both outputs from one acquisition — panorama and data carrier.

Inventory path and its range of validity. On plot-wide TLS clouds the chain uses 3DFin [11] as stem detector. On two manually labelled plots of the SegmentedForests dataset [12] it reaches 98.5 % recall / 95.7 % precision (plot_06, 68 stems) and 94.5 % / 100 % (plot_07, 128 stems), clearly ahead of the simpler detectors we also tested. This accuracy depends on circumferential stem coverage: 3DFin accepts a diameter only if the stem cross-section at breast height is covered in at least 9 of 16 sectors (≈ 202°). The labelled plots offer a median arc of 280–360°; on the Renon stand, merged from four nearby scan positions, the median is 180°, and 3DFin declines a diameter for 29 of 37 detected stems — correct behaviour, since the data do not support one. The inventory markers are therefore only as reliable as the scan geometry behind them. A systematic comparison of detectors as a function of coverage is beyond the scope of this software paper; per-method results and scripts are documented in the repository (`scripts/BENCH_DETECTION.md`).

Consumer/DSLR fisheye → scene (stitching). Six overlapping fisheye frames are stitched automatically and published as a navigable scene, demonstrating the pose-unknown branch across the low-cost end of the device spectrum.

Quantitative comparison against CC0 ground truth. From each of eleven CC0 reference panoramas (Poly Haven, 8192×4096) we render synthetic captures and rebuild a panorama (2048×1024) — once by stitching six 180° fisheye frames (pose withheld), once by reprojecting six 90° pinhole frames plus zenith/nadir (pose used) — and measure both against the same original (PSNR, SSIM [8]).

Counting convention (used throughout): **completed** means the chain wrote a panorama at all; **usable** additionally means the result is geometrically correct. The distinction matters for stitching, where two runs produce a complete but misregistered image.

Branch	completed	usable	PSNR (dB)	SSIM	seam offset (px)	runtime
Stitching (pose estimated)	8 / 11	**6 / 11**	22.98 ± 2.74	0.837 ± 0.060	0.14 (p95 0.53)	14.6 s
Reprojection (pose known)	11 / 11	**11 / 11**	**26.87 ± 1.96**	**0.886 ± 0.039**	**0.10 (p95 0.23)**	2.1 s

Aggregate figures refer to the *usable* runs; the two misregistered stitches (8.6 and 11.7 dB) are excluded because they would otherwise dominate mean and spread, and are reported separately below.

The pose-based branch is better on every measure (+3.9 dB, +0.05 SSIM, a third less local displacement) and reconstructs all eleven scenes, while stitching fails on five: three times `enblend` aborts (excessive overlap, degenerate mask geometry, no detectable seam) and twice a geometrically wrong panorama results (confirmed under exhaustive search over rotation, tilt, and mirroring, staying below 11.7 dB).

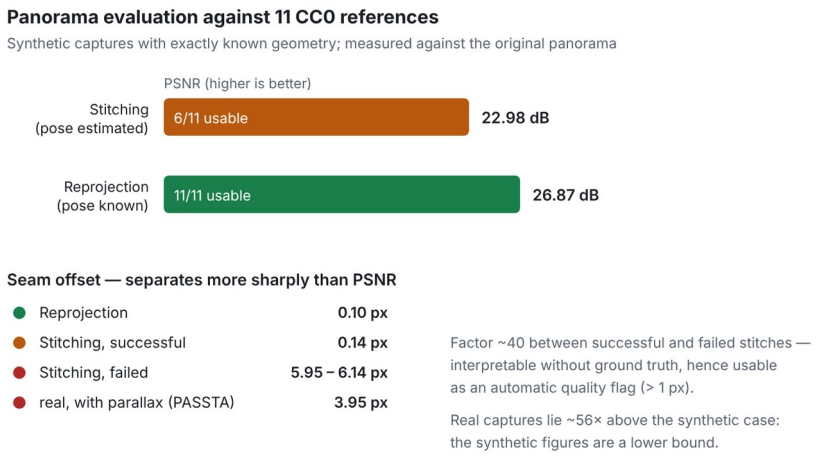


Figure 2. *Panorama evaluation against eleven CC0 references. The pose-based branch is more accurate and reconstructs every scene; the seam offset separates good from failed reconstructions by a factor of ~40 and is therefore usable as a ground-truth-free quality flag. Real captures with parallax lie ~56× above the synthetic case, which makes the synthetic figures a lower bound.*

Seam offset as an automatic quality flag. Block-wise phase correlation [9] against the reference (64-px blocks, 50% overlap, low-structure blocks discarded) separates good from failed reconstructions more sharply than PSNR or SSIM: median seam offset 0.11–0.19 px (good stitches) and 0.07–0.13 px (reprojection) versus 5.95–6.14 px (failed stitches) — a factor of ~40, interpretable without knowing the ground truth of a single scene, and thus a robust threshold (here > 1 px median) for a GUI quality flag.

Real captures: the actual cost of parallax

The caveat that the synthetic figures are upper bounds can be resolved with open data. `eval_seams.py` measures seam offset **without a reference panorama**: `nona` remaps each capture separately into the panorama canvas, and in the overlaps two source images show the same viewing direction — with perfect registration and rotation about the nodal point they would be identical there. Their local displacement is the error.

The real capture is **PASSTA LunchRoom** [10] (Zenodo, CC BY 4.0): 72 photographs from a rotating Canon EOS 70D with supplied calibration. As a control, synthetic captures with **identical geometry** are rendered from a CC0 panorama — 18 positions, 93.6° rectilinear, same stitching chain, same canvas width. The only difference is that the real captures carry genuine parallax, sensor noise, and exposure differences:

	synthetic (parallax-free)	real (rotating camera)
seam offset, median	**0.07 px**	**3.95 px**
p95	0.18 px	24.78 px
maximum	0.48 px	29.99 px
blocks above 1 px	0.0 %	67.5 %

The median is ~56× higher and the 95th percentile ~140×. This substantiates what could previously only be asserted: the synthetic evaluation understates the real stitching error by orders of magnitude, and the gap measured there between the two branches is a **lower bound**.

Interpretation. PASSTA LunchRoom is an indoor scene with nearby objects, and parallax is worst at short range. An outdoor stand with predominantly distant objects will fare better; the 3.95 px is therefore an example of the unfavourable case, not a universal value. Part of the difference also stems from noise and exposure variation rather than parallax alone. The pose-based branch is unaffected by all of this: it has no overlap seams, because every viewing direction comes from exactly one camera.

Honesty of the numbers. All synthetic captures share one nodal point; the PSNR/SSIM stitching figures are therefore **upper bounds**, and the section above quantifies the distance to reality (~56× in median seam offset). No sensor noise, exposure differences, or distortion are simulated — the geometry of the chain is measured, not camera image quality. Absolute PSNR is capped by the resolution chain and is meaningful only in the comparison of the two branches. The reprojection figures hold only since switching to bilinear sampling; with nearest-neighbour sampling the same branch scored 25.4 dB / SSIM 0.871 instead of 28.2 / 0.935 (ph-mossy-forest) — an implementation artefact that could have been reported as a property of the method.

Runtime per input class, end to end. Measured through the running containerized service — upload, queue, processing, and publication included — with the input type *not* declared, so the chain detects it itself:

Input class	captures	input	upload	processing	**total**
Consumer	1	2.5 MB	0.2 s	2.0 s	**2.2 s**

Input class	captures	input	upload	processing	**total**
360° (finished equirect)					
DSLR/fisheye (stitching)	18	6.9 MB	0.4 s	69.3 s	**69.7 s**
TLS scan (E57, reprojection)	1	177.9 MB	4.8 s	27.7 s	**32.5 s**

(Medians of two runs each, consumer CPU.) The input class was detected correctly in all six runs — an independent confirmation of the dispatch described in Section 2.1. Note that the most expensive case is not the largest dataset: the 178 MB scan finishes in 32 s while 6.9 MB of individual frames take 70 s. Cost comes from *registration*, not data volume — precisely what pose-based reprojection avoids. No manual intervention was required in any class.

4. Impact

The tool makes reproducible, self-hosted 360°/3D documentation accessible without vendor lock-in and across the full device–cost spectrum. Two contributions are of wider use. First, it turns the routinely discarded, pose-bearing scanner RGB images into first-class navigable panoramas and data carriers; the evaluation quantifies why this is preferable when pose is available (seamless, ~7× faster, no stitching failures). Second, the seam-offset metric provides a ground-truth-free quality signal that can gate automated batch processing. For environmental and forest monitoring specifically, the enrichment path couples a walkable visual record with quantitative inventory on the same acquisition, supporting communication and revisitation of plots.

5. Conclusions

We presented an open-source, containerized, GUI-driven chain that converts multi-image captures — with or without known pose — into walkable web 3D scenes, self-hostable end to end. A ground-truth evaluation shows the pose-based reprojection branch to be more accurate, more robust, and faster than stitching when scanner poses are available, and yields an interpretable automatic quality flag for the stitching branch. Future work: wire the quality flag into the GUI; integrate further representations (3D Gaussian splatting, meshes) in the same viewer; obtain a freely licensed set of raw dual-fisheye frames from a consumer 360° camera, which would close the last gap in device coverage for the stitching branch; and run the prepared usability study (System Usability Scale [13, 14]).

Data availability

All data used in this work are third-party open data. Terrestrial laser scans: Renon / ICOS IT-Ren (Leica BLK360, E57; CC BY 4.0) [15] and SegmentedForests (MIT) [12]. Reference panoramas for the quantitative evaluation: eleven Poly Haven HDRIs (CC0, https://polyhaven.com). Real multi-image capture: PASSTA LunchRoom (CC BY 4.0) [10].

Source code, container recipe and evaluation scripts: https://github.com/anayana/For3Dsuite (GPL-3.0-or-later), archived at https://doi.org/10.5281/zenodo.23323560.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Angel Naya: Conceptualization, Methodology, Software, Validation, Formal analysis, Data curation, Visualization, Writing – original draft, Writing – review & editing.

Acknowledgements

The author thanks the providers of the open datasets used here: the ICOS ecosystem station IT-Ren (Renon) and Chang et al. for the Renon laser scans, the authors of SegmentedForests and PASSTA, and the Poly Haven contributors. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

References

[1] Brown, M., Lowe, D. G. (2007). Automatic panoramic image stitching using invariant features. *International Journal of Computer Vision* 74(1), 59–73. https://doi.org/10.1007/s11263-006-0002-3

[2] Szeliski, R. (2006). Image alignment and stitching: a tutorial. *Foundations and Trends in Computer Graphics and Vision* 2(1), 1–104. https://doi.org/10.1561/0600000009

[3] Petroff, M. A. (2019). Pannellum: a lightweight web-based panorama viewer. *Journal of Open Source Software* 4(40), 1628. https://doi.org/10.21105/joss.01628

[4] Schütz, M. (2016). *Potree: Rendering large point clouds in web browsers.* Diploma thesis, TU Wien.

[5] Huber, D. (2011). The ASTM E57 file format for 3D imaging data exchange. *Proc. SPIE* 7864, Three-Dimensional Imaging, Interaction, and Measurement, 78640A. https://doi.org/10.1117/12.876555

[6] Houshiar, H., Elseberg, J., Borrmann, D., Nüchter, A. (2015). A study of projections for key point based registration of panoramic terrestrial 3D laser scan. *Geo-spatial Information Science* 18(1), 11–31. https://doi.org/10.1080/10095020.2015.1017913

[7] Kang, Z., et al. (2009). Automatic registration of terrestrial laser scanning point clouds using panoramic reflectance images. *Sensors* 9(4), 2621–2646. https://doi.org/10.3390/s90402621

[8] Wang, Z., Bovik, A. C., Sheikh, H. R., Simoncelli, E. P. (2004). Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* 13(4), 600–612. https://doi.org/10.1109/TIP.2003.819861

[9] Kuglin, C. D., Hines, D. C. (1975). The phase correlation image alignment method. *Proc. IEEE Int. Conf. on Cybernetics and Society*, 163–165.

[10] Meneghetti, G., Danelljan, M., Felsberg, M. (2015). Image alignment for panorama stitching in sparsely structured environments [Dataset]. Zenodo. CC BY 4.0. https://doi.org/10.5281/zenodo.19663081

[11] Laino, D., Cabo, C., Prendes, C., et al. (2024). 3DFin: a software for automated 3D forest inventories from terrestrial point clouds. *Forestry* 97(4), 479–496. https://doi.org/10.1093/forestry/cpae020

[12] Laino, D., Cabo, C., Ordóñez, C., et al. (2026). SegmentedForests: a labelled dataset of terrestrial LiDAR point clouds for semantic segmentation of forests. *Forestry* 99(2). https://doi.org/10.1093/forestry/cpaf062 · Data: https://doi.org/10.5281/zenodo.17396681

[13] Brooke, J. (1996). SUS: a “quick and dirty” usability scale. In: *Usability Evaluation in Industry*, Taylor & Francis, 189–194.

[14] Sauro, J., Lewis, J. R. (2016). *Quantifying the User Experience: Practical Statistics for User Research*, 2nd ed., Morgan Kaufmann.

[15] Chang, M., Candotti, A., von Ellenrieder, K., Tomelleri, E., Camurri, M. (2025). Data for: “A multi-platform LiDAR dataset for standardized forest inventory measurement at long term ecological monitoring sites” [Dataset]. Zenodo. https://doi.org/10.5281/zenodo.17186174

Software used, with the versions that produced the reported figures: Hugin/Panotools 2024.0.1 (GPL-2.0), Pannellum 2.5.6 (MIT), three.js r160 (MIT), Leaflet 1.9.4 (BSD-2), Caddy 2 (Apache-2.0), Garage 1.0.1 (AGPL-3.0), FastAPI (MIT), NumPy 1.26.4, Pillow 12.3.0, OpenCV 5.0.0, laspy 2.5.4, 3DFin/dendromatics, TreeGrOSS (GPL-3.0, isolated as a separate process).

Checked against publisher records (Oct 2026): [1]–[3], [5]–[7], [10]–[12], [15].